

Real-Time Systems / Real-Time Operating Systems

ECE445M/ECE380L.12, Spring 2024

Final Exam Solutions

Date: May 4, 2024

UT EID: _____

Printed Name: _____
Last, First

Your signature is your promise that you have not cheated and will not cheat on this exam, nor will you help others to cheat on this exam:

Signature: _____

Instructions:

- Open book, open notes and open web.
- No electronic devices other than your laptop/PC (cell phones off and stowed away).
- You are allowed to access any resource on the internet, but no electronic communication other than with instructors.
- Please be sure that your answers to all questions (and all supporting work that is required) are contained in the space (boxes) provided.
- For all questions, unless otherwise stated, find the most efficient (time, resources) solution.

Problem 1	20	
Problem 2	20	
Problem 3	20	
Problem 4	20	
Problem 5	20+10	
Total	100+10	

Name: _____

Problem 1 (20 points): Miscellaneous

a) Why can an SVC call not use OS_Wait()?

Calling OS_Wait() from an interrupt handler can lock or crash the system. In a spinlock semaphore implementation, the interrupt handler may never exit since no foreground thread can run to release the semaphore. In a blocking semaphore implementation, this may try to trigger a context switch from within the interrupt handler, where either the context switch will not run until the SVC_Handler is exited breaking OS_Wait() semantics (continues without holding the semaphore) or a context switch from within an ISR is attempted (with issues as outlined in the midterm).

b) Should a remote interpreter use a TCP or UDP network connection and why?

A remote interpreter requires a reliable connection with in-order delivery, so needs to use TCP. In UDP, data can get lost and can arrive out of order.

c) Explain the difference between external and internal fragmentation? How can you reduce the amount of external and internal fragmentation in a file system?

External fragmentation occurs if the remaining free space on the disk can not be all allocated to the same file. To reduce external fragmentation, one can defragment the disk or switch to an allocation scheme that does not limit which free blocks can belong to the same file (e.g. linked allocation).

Internal fragmentation occurs when space on the disk is allocated in fixed-size chunks (blocks) and a file does not occupy all space in each of its block, i.e. some bytes in a block are unused. This can be reduced by making blocks smaller, but this creates other overhead for meta-data.

d) What additional hardware is required to implement virtual memory? Why is this hardware needed for virtual memory?

To implement virtual memory, hardware support is needed to realized address translation from virtual to physical addresses of every load and store. This is usually done in a memory management unit (MMU) that can then also implementation other features such as memory protections and other access checks.

Name: _____

Problem 2 (20 points): CAN Networking

Given a CAN bus with 3 devices each trying to send a message with a standard 11 bit ID:

Device	ID
Device 1	0b00110101100
Device 2	0b00110010011
Device 3	0b01000000000

- a) Fill out the CAN trace for the first frame that would end up on the bus and label all the bits. Some of the sections and labels are filled in for you. Hint: note that CAN applies bit stuffing for all bits in the frame, where one stuffed bit is pre-marked below.

Label	SOF	ID											RTR	IDE	r0	DLC	
Bit #	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Value	0	0	0	1	1	0	0	1	0	0	1	1	0	0	0	0	0

Label	Stuff	DLC (cont.)				Data								Stuff	CRC			
Bit #	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34
Value	1	0	1	0	1	1	0	0	0	0	0	1	1	1	0	1	0	0

Label	CRC (cont.)										Ack		EOF				
Bit #	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51
Value	0	1	1	0	1	0	0	0	1	1	0	1	1	1	1	1	1

- b) Which device was transmitting the first frame here, and what was the data payload?

Device 2 transmitted the frame since the message it is sending has the lowest ID.

The payload is a single byte with value (after unstuffing) of 0b01100000.

Name: _____

Problem 3 (20 points): File System

Given the initial state of a FAT filesystem and the list of accesses made starting from this state below. Assume each block is 512 bytes. Assume that blocks are grabbed from and returned back to the front of the free list on allocation and deallocation, respectively.

a) Show the state of the directory and the FAT at the end of the access sequence.

File Accesses:

1. Create File A
2. Write 700 Bytes to File A
3. Append 200 Bytes to File A
4. Append 100 Bytes to File A
5. Create File B
6. Create File C
7. Write 800 Bytes to File C
8. Write 300 Bytes to File B
9. Append 100 Bytes to File A
10. Delete File C
11. Create File D
12. Append 100 Bytes to File A
13. Write 1800 Bytes to File D
14. Append 1000 Bytes to File A

Directory:

File Name	Starting Block	File Size
X	3	2,042 bytes
Y	2	256 bytes
A	1	2,200 bytes
B	9	300 bytes
D	6	1,800 bytes
Free	16	2,560 bytes

FAT	Initial Data	New Data
0	X	X
1	5	5
2	-	
3	4	4
4	8	8
5	6	11
6	7	7
7	9	12
8	10	10
9	11	-
10	-	
11	12	14
12	13	13
13	14	-
14	15	15
15	16	-
16	17	17
17	18	18
18	19	19
19	20	20
20	-	-

b) What is the internal and external fragmentation on the disk at the end of the sequence?

Internal fragmentation: 6 bytes in file X + 256 bytes in file Y + 360 bytes in file A + 212 bytes in file B + 248 bytes in file D = 1,082 unused bytes in allocated blocks. In addition, one block (512 bytes) is used to store directory and FAT.

External fragmentation: none in a FAT file system

Name: _____

- c) In many flash devices, the number of times a particular block on disk can be written to is limited, where wear of the device causing failures is a serious problem. One way to help prevent wear is to implement wear-leveling. Explain how you could modify the current file system implementation to minimize wear?

A simple form of wear leveling can be achieved by allocating blocks from one end of the free list while returning them to the other end.

More complex approaches will also keep a count of how many times a block is written and dynamically remap blocks used by a file to equalized write counts across all blocks on a disk.

Scratch space for Problem 3(a):

Name: _____

Problem 4 (20 points): Heap

- a) Assume a 80 byte heap organized in 4-byte words (int32) with headers and tails for each block indicating block size (in words) with negative size indicating free space. Given the sequence of *Heap_Malloc()* and *Heap_Free()* calls, show the final state of each heap and indicate if the sequence of allocations is successful. Assume the heap is initially empty.

```

A = Heap_Malloc(10);
B = Heap_Malloc(4);
C = Heap_Malloc(2);
D = Heap_Malloc(2);
Heap_Free(A);
Heap_Free(C);
E = Heap_Malloc(3);
F = Heap_Malloc(15);

```

Initial Heap	First Fit	Best Fit	Worst Fit
-18	+1	-3	-3
	<i>E</i>		
	+1		
	-0		
	-0	-3	-3
	+1	+1	+1
	<i>B</i>	<i>B</i>	<i>B</i>
	+1	+1	+1
	-1	+1	-1
		<i>E</i>	
	-1	+1	-1
	+1	+1	+1
	<i>D</i>	<i>D</i>	<i>D</i>
	+1	+1	+1
	+4	+4	+1
	<i>F</i>	<i>F</i>	<i>E</i>
	<i>F</i>	<i>F</i>	+1
	<i>F</i>	<i>F</i>	-1
	<i>F</i>	<i>F</i>	
-18	+4	+4	-1

Heap_Malloc for F fails.

Name: _____

- b) For each of the 3 different heap algorithms and the sequence of allocations in part a), how much internal and external fragmentation has each heap allocation scheme at the end of the sequence or at the point where the sequence fails? Which heap allocation scheme had the least amount of external fragmentation (size of the largest allocatable block vs. total free space)?

First fit:

Internal fragmentation = 2 bytes for D + 1 byte for E + 1 byte for F = 4 bytes

In addition, $12 \times 4 = 48$ bytes overhead for header/tail information

External fragmentation = $1 - (1\text{-word block allocatable} / 1 \text{ word free}) = 0\%$

Best fit:

Internal fragmentation = 2 bytes for D + 1 byte for E + 1 byte for F = 4 bytes

In addition, $10 \times 4 = 40$ bytes overhead for header/tail information

External fragmentation = $1 - (3\text{-word block allocatable} / 3 \text{ words free}) = 0\%$

Worst fit:

Internal fragmentation = 2 bytes for D + 1 byte for E = 3 bytes

In addition, $12 \times 4 = 48$ bytes overhead for header/tail information

External fragmentation = $1 - (3\text{-word block allocatable} / 5 \text{ words free}) = 40\%$

- c) If any of the allocation schemes fail, is it possible to rearrange the order of mallocs and free such that all of the allocation schemes are successful? If so, provide the new order, or explain why it is not possible.

Re-order the initial Heap_Malloc() sequence such that A and C are allocated last, and hence in consecutive blocks on the heap followed by free space. Such that when they are freed again, they will be merged with the free space into one large free block independent of the scheme.

```
B = Heap_Malloc(4);
D = Heap_Malloc(2);
A = Heap_Malloc(10);
C = Heap_Malloc(2);
Heap_Free(A);
Heap_Free(C);
E = Heap_Malloc(3);
F = Heap_Malloc(15);
```

Name: _____

Problem 5 (20 points + 10 points bonus): Real-Time System Design

A group of Aggies are attempting to develop their own Formula 0001 race car with the following components:

- An external sensor that brings in data that, based on the maximum car speed, changes at a maximum rate of 125Hz.
- A sensor sampling task that reads one sample from the ADC and processes it taking 1ms every time it executes.
- A PID controller that takes 1 ms every time it executes and controls a motor with a time constant of 40ms.
- A network task that receives 18 bytes of data from the network and places them into a queue taking 2ms every time it runs, where network data arrives at a rate of 3,000 bytes/s.
- A logging task that removes 24 bytes of data from the queue and writes it to disk taking 1ms every time it runs. The SD card has a maximum bandwidth of 100 kbytes/s.

The Aggies implemented this system by running a rate monotonic scheduler with the following task periods and priorities below:

Task	Duration	Period	Priority
Logging	1 ms	12 ms	1
Control PID	1 ms	2 ms	2
ADC Sampling	1 ms	5 ms	3
Network	2 ms	8 ms	4

- a) Does this system work or did the Aggies screw up their implementation? What are the bugs with the current implementation?

The Aggie system has the following bugs:

1) Sensor sampling rate is too low, it violates Nyquist constraints. The sampling task needs to run at a rate of at least 250Hz, i.e. once every 4ms.

2) The control task runs faster than the sampling task. It should run at the same rate of 4ms (which is also 10 times faster than the motor time constant).

3) The network task runs too infrequently to process data at the rate it is arriving over the network. The network task needs to process 3,000 bytes per second, i.e. 18 bytes every 6ms.

*4) The logging task consumes data slower than the network task produces it. Data production and consumption rates should match in order to not overflow the queue. With the network task producing 18 bytes every 6ms, the logging task needs to consume 24 bytes at least every 8ms (or with the original network task period of 8ms, the logging task would need to run every $8/18 * 24 = 10.67\text{ms}$).*

5) RMS priorities are assigned wrong. They should be 4,1,2,3.

6) The system is overloaded. Total system utilization is $1/12 + 1/2 + 1/5 + 2/8 = 103.33\%$ without even accounting for other OS overheads such as context switch times.

Name: _____

- b) **(Extra Credit)** Modify the system into a working solution that does not have any issues and show the final system schedule below. You may not change the tasks or the external robot hardware.

See above, the task set needs to run with the following periods and priorities:

Task	Duration	Period	Priority
Logging	1 ms	8 ms	4
Control PID	1 ms	4 ms	1
ADC Sampling	1 ms	4 ms	2
Network	2 ms	6 ms	3

The utilization of this task set is $1/8 + 2/4 + 2/6 \approx 96\%$

This is above the RMS limit, and also not schedulable with RMS in the critical instant, where the Logging task would miss its deadline (shown below). To run this taskset, an EDF scheduler is needed (shown in green below).

